XChain 머클 증명 및 검증 시스템 분석

개요

Omni 네트워크의 크로스체인 메시지 검증은 머클 트리(Merkle Tree) 기반의 암호학적 증명 시스템을 사용합니다. 이 시스템은 크로스체인 메시지의 무결성을 보장하고, 효율적인 다중 메시지 검증을 가능하게 하는 핵심 보안 메커니즘입니다.

머클 트리 라이브러리 (`lib/merkle`)

기본 구조

Omni의 머클 트리 구현은 OpenZeppelin의 JavaScript merkle-tree 라이브러리를 Go로 포팅한 것입니다.

```
// 도메인 분리 태그 - 리프의 도메인을 정의

type DomainSeparationTag byte



// 표준 리프 해시 - 이중 해싱으로 사전 이미지 공격 방지

func  StdLeafHash(dst DomainSeparationTag, data []byte) [32]byte {

h1 := hash(append([]byte{byte(dst)}, data...))

h2 := hash(h1[:])

return  h2

}
```

트리 생성 알고리즘

```go
func MakeTree(leaves [][32]byte) ([][32]byte, error) {
if len(leaves) == 0 {
return nil, errors.New("no leaves provided")
}

treeLen := 2*len(leaves) - 1
tree := make([][32]byte, treeLen)

// 리프를 역순으로 채움
for i, leaf := range leaves {
tree[treeLen-1-i] = leaf
}

// 루트까지 중간 노드들을 채움
for i := treeLen - 1 - len(leaves); i >= 0; i-- {
tree[i] = hashPair(tree[leftChildIndex(i)], tree[rightChildIndex(i)])
}

return tree, nil
}
```

트리 구조 특징:

- **완전 이진 트리**: 모든 내부 노드는 정확히 2개의 자식을 가짐
- **홀수 노드**: 트리는 항상 홀수 개의 노드를 가짐 (2n-1)
- **역순 저장**: 리프가 배열의 끝부분에 저장됨
- **인덱스 0**: 루트 노드가 인덱스 0에 위치

다중 증명 (MultiProof)

```
type MultiProof struct {
Leaves [][32]byte  // 증명할 리프들
Proof [][32]byte  // 증명 노드들
ProofFlags []bool  // 증명 플래그 (형제 노드 여부)
}

func  GetMultiProof(tree [][32]byte, indices ...int) (MultiProof, error)
```

다중 증명 알고리즘:

1. **인덱스 정렬**: 역순으로 정렬하여 처리
2. **중복 검사**: 동일한 인덱스 중복 방지
3. **스택 기반 처리**: 상향식으로 증명 경로 구성
4. **플래그 생성**: 형제 노드 포함 여부 표시

해시 함수

```
// 쌍 해시 - 정렬된 연결 후 해싱
```

```go
func hashPair(a [32]byte, b [32]byte) [32]byte {
return hash(concatBytes(sortBytes(a[:], b[:])...))
}

// 기본 해시 - Keccak256 사용
func hash(buf []byte) [32]byte {
return cast.Must32(crypto.Keccak256(buf))
}
```

XChain 머클 시스템 (`lib/xchain/merkle.go`)

도메인 분리 태그

```go
const (
DSTUnknown merkle.DomainSeparationTag = 0  // 알 수 없음
DSTHeader merkle.DomainSeparationTag = 1  // 헤더용
DSTMessage merkle.DomainSeparationTag = 2  // 메시지용
)
```

도메인 분리의 목적:

- **타입 안전성**: 서로 다른 데이터 타입 간 해시 충돌 방지
- **보안 강화**: 크로스 도메인 공격 방지
- **명확한 구분**: 헤더와 메시지 데이터의 명확한 분리

메시지 트리 (MsgTree)

```go
type MsgTree struct {
tree [][32]byte  // 모든 노드를 포함한 머클 트리
indices  map[[32]byte]int  // 노드 인덱스 역방향 조회 테이블
}
```

주요 메서드:

1. 트리 생성

```go
func  NewMsgTree(msgs []Msg) (MsgTree, error) {
// 메시지 순서 검증 (로그 인덱스 기준)
for  i := 1;  i < len(msgs);  i++ {
if  msgs[i-1].LogIndex >= msgs[i].LogIndex {
return MsgTree{}, errors.New("messages not ordered by log index [BUG]")
}
}

// 메시지를 리프로 변환
leafs := make([][32]byte, 0, len(msgs))
for  _, msg := range  msgs {
msgLeaf, err := msgLeaf(msg)
if  err != nil {
return MsgTree{}, err
```

```go
}
leafs = append(leafs, msgLeaf)
}

// 머클 트리 생성
tree, err := merkle.MakeTree(leafs)
if err != nil {
return MsgTree{}, err
}

// 역방향 조회 테이블 생성
indices := make(map[[32]byte]int)
for i, node := range tree {
indices[node] = i
}

return MsgTree{tree: tree, indices: indices}, nil
}
```

2. 메시지 루트 조회

```go
func (t MsgTree) MsgRoot() [32]byte {
return t.tree[0] // 루트 노드 반환
```

```
}
```

3. 다중 증명 생성

```
func (t MsgTree) Proof(msgs []Msg) (merkle.MultiProof, error) {
// 증명할 인덱스들 수집
indices := make([]int, 0, len(msgs))
for _, msg := range msgs {
msgLeaf, err := msgLeaf(msg)
if err != nil {
return merkle.MultiProof{}, err
}
msgIndex, err := t.leafIndex(msgLeaf)
if err != nil {
return merkle.MultiProof{}, errors.Wrap(err, "msg index")
}
indices = append(indices, msgIndex)
}

return merkle.GetMultiProof(t.tree, indices...)
}
```

메시지 리프 생성

```go
func msgLeaf(msg Msg) ([32]byte, error) {
bz, err := encodeMsg(msg)
if err != nil {
return [32]byte{}, errors.Wrap(err, "encode message")
}

return merkle.StdLeafHash(DSTMessage, bz), nil
}
```

메시지 인코딩 구조:

```go
msgABI = mustABITuple([]abi.ArgumentMarshaling{
{Name: "DestChainID", Type: typUint64},
{Name: "ShardID", Type: typUint64},
{Name: "StreamOffset", Type: typUint64},
{Name: "SourceMsgSender", Type: typAddress},
{Name: "DestAddress", Type: typAddress},
{Name: "Data", Type: typBytes},
{Name: "DestGasLimit", Type: typUint64},
})
```

증명 루트 시스템

제출 헤더 (Submission Header)

```
type submissionHeader struct {
SourceChainID uint64 // 소스 체인 ID
ConsensusChainID uint64 // 합의 체인 ID
ConfLevel ConfLevel // 확인 레벨
AttestOffset uint64 // 증명 오프셋
BlockHeight uint64 // 블록 높이
BlockHash common.Hash // 블록 해시
}
```

증명 루트 계산

```
func AttestationRoot(attHeader AttestHeader, blockHeader BlockHeader, msgRoot c
// 헤더 리프 생성
headerLeaf, err := submissionHeaderLeaf(attHeader, blockHeader)
if err != nil {
return [32]byte{}, err
}

// 2-리프 트리 생성 (메시지 루트 + 헤더 리프)
tree, err := merkle.MakeTree([][32]byte{msgRoot, headerLeaf})
if err != nil {
```

```
return [32]byte{}, err
}

return tree[0], nil // 루트 반환
}
```

증명 루트 구조:

```
AttestationRoot
/ \
MsgRoot HeaderLeaf
```

헤더 리프 생성

```
func submissionHeaderLeaf(attHeader AttestHeader, blockHeader BlockHeader) ([32
bz, err := encodeSubmissionHeader(attHeader, blockHeader)
if err != nil {
return [32]byte{}, errors.Wrap(err, "encode block submissionHeader")
}

return merkle.StdLeafHash(DSTHeader, bz), nil
}
```

ABI 인코딩 시스템 (`lib/xchain/abi.go`)

메시지 인코딩

```
func encodeMsg(msg Msg) ([]byte, error) {
resp, err := msgABI.Pack(msg)
if err != nil {
return nil, errors.Wrap(err, "pack xchain msg")
}
return resp, nil
}
```

제출 헤더 인코딩

```
func encodeSubmissionHeader(attHeader AttestHeader, blockHeader BlockHeader) ([
if attHeader.ChainVersion.ID != blockHeader.ChainID {
return nil, errors.New("chain ID mismatch")
}

resp, err := submissionHeaderABI.Pack(submissionHeader{
SourceChainID: attHeader.ChainVersion.ID,
ConsensusChainID: attHeader.ConsensusChainID,
ConfLevel: attHeader.ChainVersion.ConfLevel,
AttestOffset: attHeader.AttestOffset,
```

```
BlockHeight: blockHeader.BlockHeight,
BlockHash: blockHeader.BlockHash,
})
if err != nil {
return nil, errors.Wrap(err, "pack xchain submissionHeader")
}


return resp, nil
}
```

제출 시스템 (Submission System)

XSubmit 인코딩/디코딩

```
// XSubmit 함수 호출 인코딩
func EncodeXSubmit(sub bindings.XSubmission) ([]byte, error) {
bytes, err := omniPortalABI.Pack("xsubmit", sub)
if err != nil {
return nil, errors.Wrap(err, "pack xsubmit")
}
return bytes, nil
}
```

```go
// XSubmit 함수 호출 디코딩
func DecodeXSubmit(txCallData []byte) (bindings.XSubmission, error) {
const method = "xsubmit"
m, ok := omniPortalABI.Methods[method]
if !ok {
return bindings.XSubmission{}, errors.New("missing method")
}

trimmed := bytes.TrimPrefix(txCallData, m.ID)
if bytes.Equal(trimmed, txCallData) {
return bindings.XSubmission{}, errors.New("tx data not prefixed with xsubmit met
}

unpacked, err := m.Inputs.Unpack(trimmed)
if err != nil {
return bindings.XSubmission{}, errors.Wrap(err, "unpack submission")
}

// ... 구조체 복사 로직
return wrap.Sub, nil
}
```

바인딩 변환

Go → Solidity 바인딩

```go
func SubmissionToBinding(sub Submission) bindings.XSubmission {
// 검증자 주소로 서명 정렬 (결정론적 순서 보장)
sort.Slice(sub.Signatures, func(i, j int) bool {
return sub.Signatures[i].ValidatorAddress.Cmp(sub.Signatures[j].ValidatorAddres
})

// 서명 변환
sigs := make([]bindings.ValidatorSigTuple, 0, len(sub.Signatures))
for _, sig := range sub.Signatures {
sigs = append(sigs, bindings.ValidatorSigTuple{
ValidatorAddr: sig.ValidatorAddress,
Signature: sig.Signature[:],
})
}

// 메시지 변환
msgs := make([]bindings.XMsg, 0, len(sub.Msgs))
for _, msg := range sub.Msgs {
msgs = append(msgs, bindings.XMsg{
DestChainId: msg.DestChainID,
ShardId: uint64(msg.ShardID),
```

```
Offset: msg.StreamOffset,

Sender: msg.SourceMsgSender,

To: msg.DestAddress,

Data: msg.Data,

GasLimit: msg.DestGasLimit,

})

}



return bindings.XSubmission{

AttestationRoot: sub.AttestationRoot,

ValidatorSetId: sub.ValidatorSetID,

BlockHeader: bindings.XBlockHeader{

SourceChainId: sub.BlockHeader.ChainID,

ConsensusChainId: sub.AttHeader.ConsensusChainID,

SourceBlockHash: sub.BlockHeader.BlockHash,

SourceBlockHeight: sub.BlockHeader.BlockHeight,

ConfLevel: uint8(sub.AttHeader.ChainVersion.ConfLevel),

Offset: sub.AttHeader.AttestOffset,

},

Proof: sub.Proof,

ProofFlags: sub.ProofFlags,

Signatures: sigs,

Msgs: msgs,
```

```
}

}
```

검증 플로우

1. 메시지 검증 과정

```
1. 메시지 수집 → 2. 순서 검증 → 3. 리프 생성 → 4. 트리 구성
                                                        ↓
8. 온체인 검증 ← 7. 제출 생성 ← 6. 증명 생성 ← 5. 루트 계산
```

단계별 상세:

1. **메시지 수집**: 블록에서 XMsg 이벤트 수집
2. **순서 검증**: LogIndex 기준 오름차순 정렬 확인
3. **리프 생성**: 각 메시지를 ABI 인코딩 후 해싱
4. **트리 구성**: 머클 트리 생성 및 인덱스 테이블 구축
5. **루트 계산**: 메시지 루트와 헤더 리프로 증명 루트 계산
6. **증명 생성**: 제출할 메시지들에 대한 다중 증명 생성
7. **제출 생성**: 증명과 서명을 포함한 제출 데이터 구성
8. **온체인 검증**: 목적지 체인에서 머클 증명 검증

2. 온체인 검증 로직

목적지 체인의 Portal 컨트랙트에서 수행되는 검증:

1. **증명 루트 검증**: 제출된 증명 루트가 저장된 것과 일치하는지 확인

2. **머클 증명 검증**: 제출된 메시지들이 메시지 루트에 포함되는지 검증

3. **서명 검증**: 검증자 서명들이 유효한지 확인

4. **쿼럼 검증**: 충분한 수의 검증자가 서명했는지 확인

보안 고려사항

1. 사전 이미지 공격 방지

```
func StdLeafHash(dst DomainSeparationTag, data []byte) [32]byte {
h1 := hash(append([]byte{byte(dst)}, data...))
h2 := hash(h1[:]) // 이중 해싱으로 사전 이미지 공격 방지
return h2
}
```

2. 도메인 분리

- **DSTHeader**: 헤더 데이터용 도메인
- **DSTMessage**: 메시지 데이터용 도메인
- 서로 다른 도메인 간 해시 충돌 방지

3. 정렬 기반 해싱

```
func hashPair(a [32]byte, b [32]byte) [32]byte {
return hash(concatBytes(sortBytes(a[:], b[:])...))
}
```

- 노드 순서에 관계없이 동일한 해시 생성
- 트리 구조의 일관성 보장

4. 메시지 순서 검증

```
for  i := 1;  i < len(msgs);  i++ {
if  msgs[i-1].LogIndex >= msgs[i].LogIndex {
return MsgTree{}, errors.New("messages not ordered by log index [BUG]")
}
}
```

- 블록 내 메시지 순서의 결정론적 보장
- 재생 공격 및 순서 조작 방지

5. 서명 정렬

```
sort.Slice(sub.Signatures, func(i, j  int) bool {
return  sub.Signatures[i].ValidatorAddress.Cmp(sub.Signatures[j].ValidatorAddres
})
```

- 검증자 주소 기준 정렬로 결정론적 순서 보장
- 서명 순서 조작 방지

이러한 머클 증명 시스템은 Omni 네트워크의 크로스체인 메시지 무결성을 암호학적으로 보장하며, 효율적인 다중 메시지 검증을 통해 확장성을 제공하는 핵심 보안 인프라입니다.